

Báo cáo tiến độ tuần

MPPI cho UAV: từ bài báo đến Gazebo và ArduPilot

Nguyễn Thành Lâm
Companion-computer local planning

13 tháng 09 năm 2026

Video demo: [Google Drive](#) Mã nguồn: [GitHub](#) – [thanhlamauto/ardu_plugin_gazebo](#)

Tóm tắt

Báo cáo trình bày cách Minařík et al. áp dụng Model Predictive Path Integral (MPPI) để điều khiển UAV, sau đó mô tả bản chuyển thể của project trong Gazebo 3D và ArduPilot SITL. Hệ thống dùng global path làm reference, point cloud LiDAR làm nguồn va chạm và gửi velocity setpoint qua MAVLink. Ba fixed-route scenario đã đạt terminal với một seed mỗi map. Phần cuối thảo luận sim-to-real, mở rộng sang PA-MPPI và các insight khi tune. Những khác biệt với paper được giữ rõ: project hiện là velocity-level adaptation, chưa phải rigid-body reproduction.

Trạng thái: 41/41 unit tests đạt offline; ba Gazebo/SITL fixed-route run đạt với một seed; RViz-click, rigid-body PA-MPPI, edge computer và hardware còn cần validation tương ứng.

1 MPPI cho UAV trong Minařík et al.

Minařík et al. đề xuất MPPI chạy hoàn toàn onboard cho UAV bay nhanh, bám trajectory và tránh vật cản [1]. Thay vì giải một bài toán gradient-based, MPPI tạo nhiều chuỗi lệnh nhiễu, mô phỏng dynamics, chấm cost rồi lấy trung bình có trọng số. Nhờ vậy collision indicator không trơn vẫn có thể được đưa trực tiếp vào objective.

Ở mỗi chu kỳ, với nominal control u_k và nhiễu ngẫu nhiên $\epsilon_{i,k}$, rollout i dùng

$$u_{i,k} = u_k + \epsilon_{i,k}, \quad x_{i,k+1} = f(x_{i,k}, u_{i,k}). \quad (1)$$

Sau khi tính tổng cost J_i , trọng số là

$$a_i = \exp\left(-\frac{J_i - J_{min}}{\lambda}\right), \quad w_i = \frac{a_i}{\sum_j a_j}. \quad (2)$$

Nominal sequence được cập nhật bằng $u_k \leftarrow u_k + \sum_i w_i \epsilon_{i,k}$. Chỉ lệnh đầu được áp dụng, sequence được dịch một bước và toàn bộ quá trình lặp lại với state mới. Đây là receding-horizon MPPI [2].

1.1 Mô hình, interface và setting của paper

Paper dùng state rigid-body $x = [p, q, v, \omega]$ gồm position, quaternion, velocity và body rate. MPPI sinh collective thrust cùng desired body rates $u = [F, \omega_x, \omega_y, \omega_z]$. Trước khi forward-simulate, tác giả quy desired body-rate change về torque và rotor thrust, clip từng rotor theo giới hạn motor, rồi dựng lại feasible thrust/body rates. Lệnh feasible được gửi tới PX4, nơi body-rate PID đóng vai trò inner loop [1].

Nhóm	Setting paper	Ý nghĩa
Compute	Jetson Orin Nano 8 GB; GPU CUDA	Parallel rollout onboard.
Controller	100 Hz; budget 10 ms	State feedback và gửi lệnh mỗi 10 ms.
Prediction	$K = 896, N = 15, \Delta t = 0.1$ s	Horizon dự đoán 1.5 s.
Interpolation	Controller step 0.01 s, prediction step 0.1 s	Tách controller rate khỏi prediction resolution.
Sampling	$\Sigma = \text{diag}(0.60, 0.15, 0.15, 0.05)$	Noise cho thrust và ba body rates.
Temperature	$\lambda = 10^{-4}$	Scale weight theo cost/units của paper.
Reference	$c_p = 400, c_q = 20, c_v = 40, c_\omega = 20$	Bám position, attitude, velocity, body rate.
Input	$R = \text{diag}(0.01, 0.05, 0.05, 0.10)$	Phạt độ lớn lệnh.
Input change	$R_\Delta = \text{diag}(0.05, 0.10, 0.10, 0.30)$	Phạt lệnh thay đổi giữa hai bước.
Collision	$c_{obs} = 10^6$	Phạt state nằm trong obstacle set.

Paper dùng MPC-based tracker để tạo smooth receding-horizon reference. Thí nghiệm thực dùng Pixhawk Cube Orange+, PX4, RTK GPS và cùng onboard computer. Các setting trên thuộc đúng platform/cost scale đó; không thể sao chép trực tiếp λ hoặc giới hạn thrust sang project khác mà bỏ qua units và dynamics.

2 Áp dụng trong Gazebo 3D và ArduPilot

Project giữ ý tưởng MPPI nhưng chủ động bắt đầu ở tầng velocity để tách bài toán local planning khỏi bài toán low-level thrust/body-rate. State vật lý là $x = [p, v, \psi]$; lệnh là $u = [v_x^{cmd}, v_y^{cmd}, v_z^{cmd}, \dot{\psi}^{cmd}]$. ArduPilot nhận lệnh bằng SET_POSITION_TARGET_LOCAL_NED trong Guided mode và vẫn đảm nhiệm attitude/motor stabilization [3, 4].

Profile chạy ba scenario dùng CPU, 10 Hz, 500 rollouts, 30 prediction steps, $\Delta t = 0.1$ s và horizon 3 s. Profile audit dùng $\lambda = 100$; giá trị này được tune cho state/control units hiện tại, không phải claim từ paper. Profile demo mượt giảm reference speed xuống 0.85 m/s và giới hạn gia tốc ngang xuống 0.9 m/s². Global path được đổi thành chuỗi reference theo thời gian (p_k^{ref}, v_k^{ref}) để cùng rollout index k so sánh cùng thời điểm.

2.1 Các hàm cost thực sự dùng

Với paper-cost profile, tổng cost của một rollout được viết đơn giản:

$$J = J_{path} + J_{vel} + J_{input} + J_{smooth} + J_{collision}. \quad (3)$$

Các thành phần được tính như sau:

$$J_{path} = 400 \sum_k \|p_k - p_k^{ref}\|^2, \quad \text{bám vị trí trên global path,} \quad (4)$$

$$J_{vel} = 40 \sum_k \|v_k - v_k^{ref}\|^2, \quad \text{bám tốc độ dọc reference,} \quad (5)$$

$$J_{input} = \sum_k u_k^T R u_k, \quad \text{phạt lệnh lớn,} \quad (6)$$

$$J_{smooth} = \sum_k (u_k - u_{k-1})^T R_\Delta (u_k - u_{k-1}), \quad \text{phạt thay đổi lệnh,} \quad (7)$$

$$J_{collision} = 10^6 \sum_k \mathbf{1}[d_k < r_{safe}], \quad \text{loại rollout đi quá gần point cloud.} \quad (8)$$

Ở đây R và R_Δ dùng các diagonal weight của paper nhưng ánh xạ sang $[v_x, v_y, v_z, \dot{\psi}]$; đây là project adaptation. d_k là khoảng cách tới điểm LiDAR gần nhất và $r_{safe} = 1.5$ m là point-cloud collision adapter. Project không dùng attitude/body-rate reference cost trong nhánh velocity-level vì state không có các biến đó. Các heuristic point-goal, soft obstacle, yaw và terminal cost của profile cũ được tắt để phép đối chiếu paper rõ ràng.

2.2 Luồng hoạt động chung khi chạy Gazebo, ArduPilot và RViz

Ba lệnh trong RUN_3_MPPI_MAPS_QUICKSTART_VI.md khác world và global path, nhưng dùng cùng runtime flow:

1. Gazebo server cập nhật physics, sensor suite phát odometry và LiDAR;
2. planner đọc state và point cloud, đổi FLU/FRD và ENU/NED đúng interface;
3. global polyline được time-parameterize thành reference trong horizon;
4. MPPI lấy mẫu 500 control sequences và forward-simulate 30 bước;
5. mỗi rollout nhận năm cost ở trên, sau đó được đổi thành exponential weight;
6. weighted noise cập nhật nominal sequence; conditioner áp low-pass, slew limit và saturation giống cả trong rollout lẫn output;
7. lệnh đầu được đổi ENU sang MAVLink Local NED rồi gửi ArduPilot;
8. ArduPilot bám velocity target; state mới quay lại planner sau 0.1 s;
9. RViz chỉ hiển thị point cloud, odometry, nominal path và top- K samples. Nếu dùng 2D Goal Pose, click mới thay route và reset warm start.

Global path chọn phía đi quanh vật cản, còn MPPI quyết định chuyển động cục bộ trong horizon. Vì vậy narrow-gate demo đi qua cửa lớn rồi vòng phía bắc hai cột; route xuyên khe nhỏ là stress-test khác, không phải cùng kết quả.

2.3 Kết quả hiện có

Ba live run dùng seed 7, Gazebo Harmonic, ArduPilot commit f808f78c và profile paper-cost trước khi tạo profile demo mượt:

Scenario	Cycles	Final error	Path	Min point clearance
Right angle	128	0.235 m	23.15 m	2.19 m
Narrow gate	192	0.230 m	35.50 m	2.07 m
Slalom	193	0.220 m	37.67 m	1.30 m

Không run nào ghi deadline miss hoặc brake event. Clearance là khoảng cách từ tâm odometry tới point LiDAR, chưa trừ footprint. Do đó một seed thành công chỉ chứng minh pipeline chạy được, chưa chứng minh robustness hoặc hardware safety.

3 Sim-to-real

Khoảng cách chính giữa simulation và quadrotor thật gồm:

- **Dynamics:** first-order velocity model không chứa motor lag, battery, drag, payload và actuator saturation đầy đủ;
- **Sensing:** LiDAR thật có noise, miss/max-range ray, motion distortion, occlusion và extrinsic calibration error;
- **Timing:** transport, estimator và compute latency làm state cũ; phải đo mean/p95/worst và deadline miss trên edge computer;
- **Frames:** ENU/NED, FLU/FRD, yaw và sensor transform phải được kiểm tra bằng motion nhỏ có dấu trước khi autonomous flight;
- **Safety:** cần geofence, manual override, stale-state hold, predictive brake, thrust/velocity limit và staged flight gate.

Paper cũng ghi nhận sai số real-world tăng ở một trajectory và quy nguyên nhân chính cho battery voltage làm thay đổi thrust-to-throttle mapping [1]. Với project này, lộ trình phù hợp là Gazebo đa seed → edge benchmark → HIL/tether → bay thật tốc độ thấp; không chuyển trực tiếp từ một video SITL sang hardware.

4 Từ MPPI sang PA-MPPI

MPPI thường tối ưu trajectory trên state và obstacle representation hiện có. PA-MPPI bổ sung nhận thức về phần môi trường chưa quan sát: map phân biệt {unknown, free, occupied}, collision cost không coi unknown là free, và perception objective ưu tiên trajectory giúp nhìn rõ goal khi goal đang bị che khuất [5]. Hiểu ngắn gọn: PA-MPPI không chỉ “né cái đã thấy”, mà còn dùng map để chọn chuyển động có ích cho việc quan sát tiếp theo.

Project đã có occupancy map ba trạng thái, ray traversal, line-of-sight và perception cost ở nhánh PA-MPPI v0. Tuy nhiên mapper hiện là prototype; unknown environment ablation và rigid-body PA-MPPI vẫn chưa được live-verified. Vì vậy chưa thể kết luận PA-MPPI tốt hơn vanilla MPPI từ ba fixed-route run hiện tại.

5 Insight khi tune MPPI

1. **Tune theo units, không chép số.** λ , noise và weights chỉ có nghĩa cùng dynamics, state scale và cost scale. Paper dùng $\lambda = 10^{-4}$; project cần 100 để tránh weight collapse.
2. **Kiểm tra hình học trước.** Nếu khe nhỏ hơn hai lần safety radius cộng footprint/uncertainty, giảm collision weight chỉ đổi “kẹt” thành “nguy cơ va chạm”.
3. **Global path quyết định homotopy.** Local MPPI horizon hữu hạn có thể không biết nên vòng trái hay phải quanh obstacle lớn; reference giúp đưa rollout vào đúng lớp đường đi.
4. **Horizon phải nhìn qua điểm quyết định.** Tăng N chỉ hữu ích khi rollout thấy cửa ra và compute vẫn kịp deadline.
5. **Noise và samples đi cùng ESS.** Noise quá nhỏ không khám phá; quá lớn tạo nhiều rollout vô ích. ESS gần 1 nghĩa update bị chi phối bởi rất ít mẫu và cần xem lại λ /cost scale.
6. **Tune feasibility trước smoothness.** Sau khi planner tìm được đường, mới giảm reference speed, acceleration limit và tăng input-change cost.
7. **Terminal cần hysteresis hợp lý.** Gate quá chặt làm UAV đã gần goal nhưng vẫn chạy optimizer; gate quá rộng báo đạt sớm.
8. **Đánh giá nhiều seed.** Giữ cùng world, path, dynamics, limits và base cost; báo cáo cả failure, clearance, path length, control smoothness và compute p95.

6 Kết luận

Paper chứng minh MPPI có thể kết hợp nonlinear dynamics, motor feasibility, reference tracking và collision cost để điều khiển UAV onboard ở 100 Hz. Project đã chuyển cấu trúc cost/reference đó sang velocity-level MPPI chạy cùng Gazebo 3D và ArduPilot, bổ sung MAVLink, frame conversion, safety gate, logging và RViz. Bước tiếp theo là xác nhận profile demo/RViz đa seed, sau đó mới đánh giá PA-MPPI unknown-space và rigid-body interface trên edge computer.

Tài liệu tham khảo

- [1] Michal Minařík, Robert Pěnička, Vojtěch Vonásek, and Martin Saska. Model predictive path integral control for agile unmanned aerial vehicles. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024. arXiv:2407.09812v1; accessed 2026-09-11.
- [2] Grady Williams, Paul Drews, Brian Goldfain, James M. Rehg, and Evangelos A. Theodorou. Information-theoretic model predictive control: Theory and applications to autonomous driving. *IEEE Transactions on Robotics*, 34(6):1603–1622, 2018. DOI: 10.1109/TRO.2018.2865891; arXiv:1707.02342.
- [3] MAVLink Project. *MAVLink Common Message Set: SET_POSITION_TARGET_LOCAL_NED and SET_ATTITUDE_TARGET*, 2026. <https://mavlink.io/en/messages/common.html>; accessed 2026-09-10. Vendored mavlink commit 71d925850d7ddb01d0e56defc1046f9d9d3bf8ae.
- [4] ArduPilot Project. *Copter Commands in Guided Mode*, 2026. <https://ardupilot.org/dev/docs/copter-commands-in-guided-mode.html>; accessed 2026-09-10.
- [5] Yifan Zhai, Rudolf Reiter, and Davide Scaramuzza. PA-MPPI: Perception-aware model predictive path integral control for quadrotor navigation in unknown environments. *IEEE Robotics and Automation Letters*, 11(3):3804–3811, 2026. DOI: 10.1109/LRA.2026.3662653; available at https://rpg.ifi.uzh.ch/docs/RAL26_Zhai.pdf.